

MF20006: Introduction to Computer Science

Lecture 2: Computer Architecture

Hui Xu

xuh@fudan.edu.cn



Outline

1. CPU and GPU

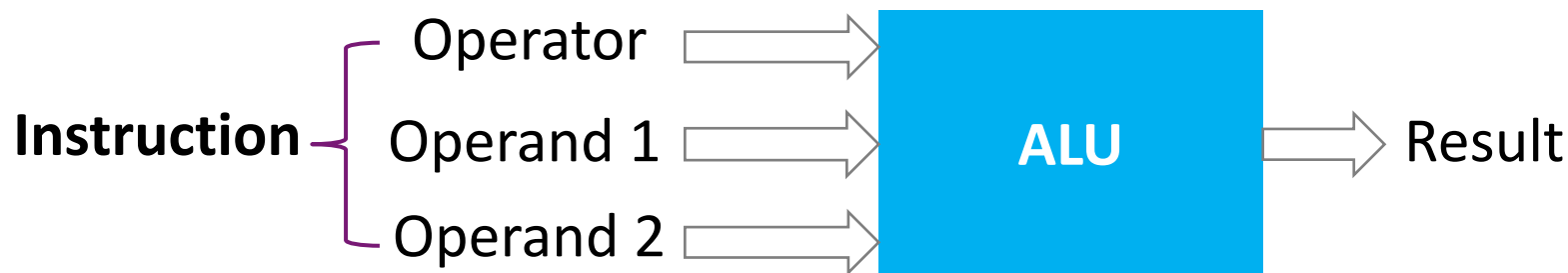
2. ISA and Assembly Code

3. I/O and More

1. CPU and GPU

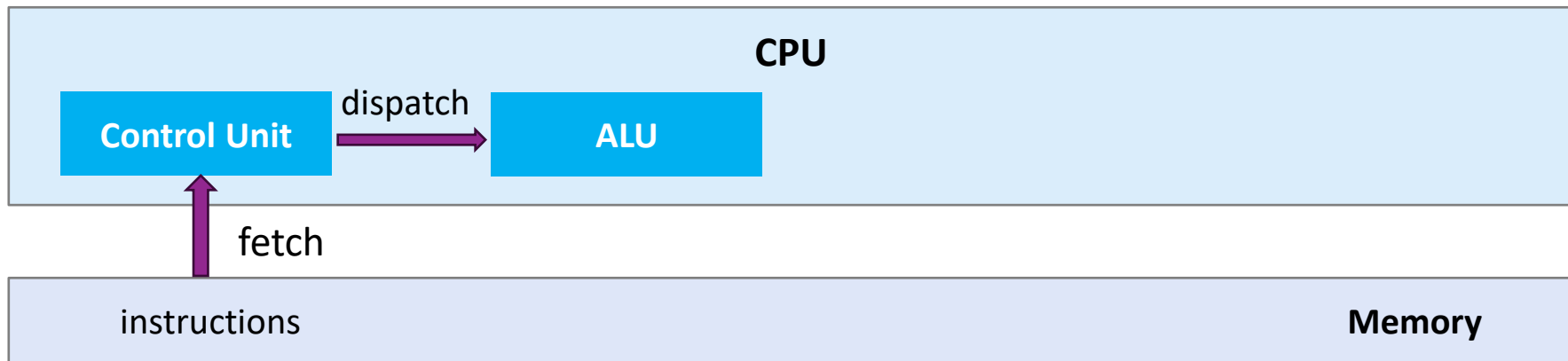
Recall: Transistor => Logic Gate => Arithmetic Logic Unit

□ **ALU: Perform arithmetic and logical operations based on the instruction.**



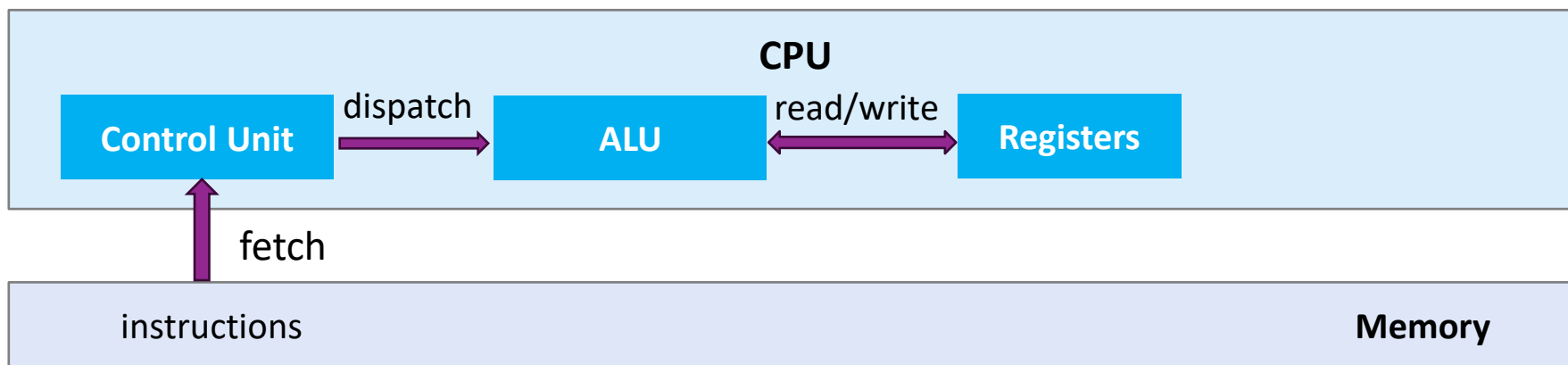
CPU: Central Processing Unit

- ❑ Arithmetic logic unit: Perform arithmetic/logic operations.
- ❑ Control unit: Fetch instructions from memory; decode and dispatch.



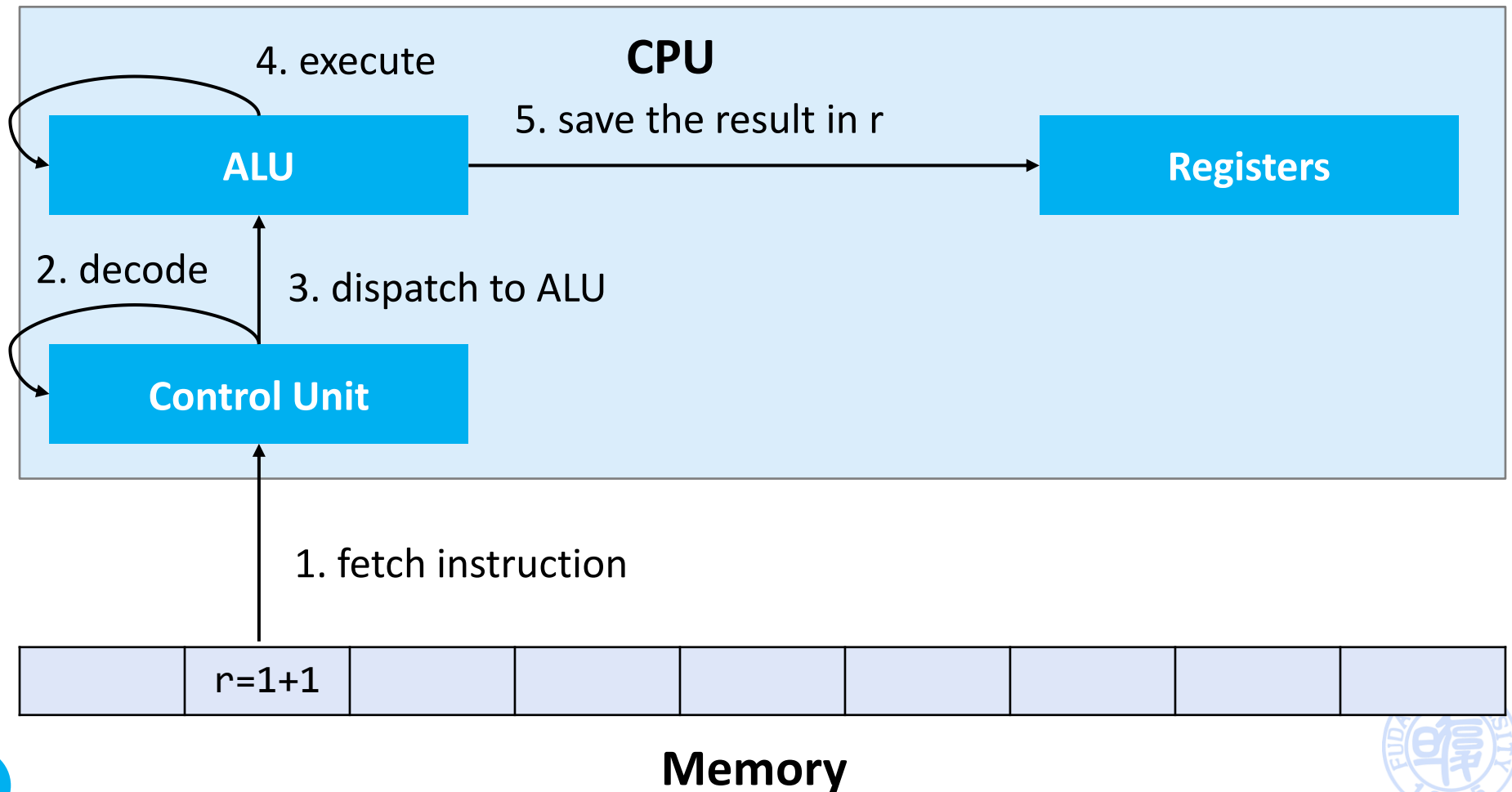
CPU: Central Processing Unit

- ❑ Arithmetic logic unit: Perform arithmetic/logic operations.
- ❑ Control unit: Fetch instructions from memory; decode and dispatch.
- ❑ **Registers: Supply operands to the ALU and save the execution result.**



Example: $r = 1 + 1$

❑ An add instruction with two instant numbers as the operands.

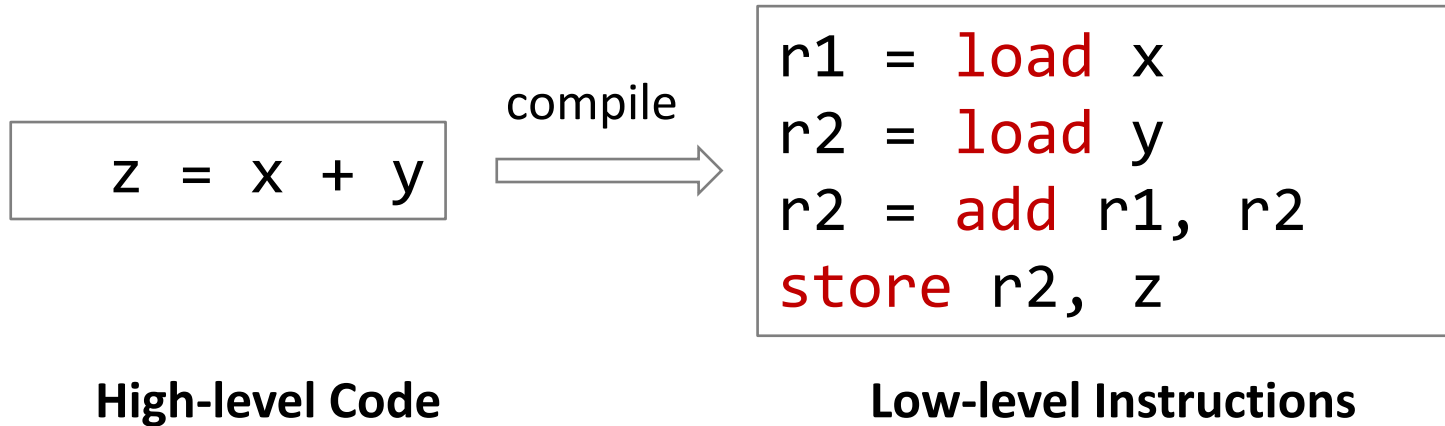


Example: $z = x + y$

❑ The operands x and y are variables.

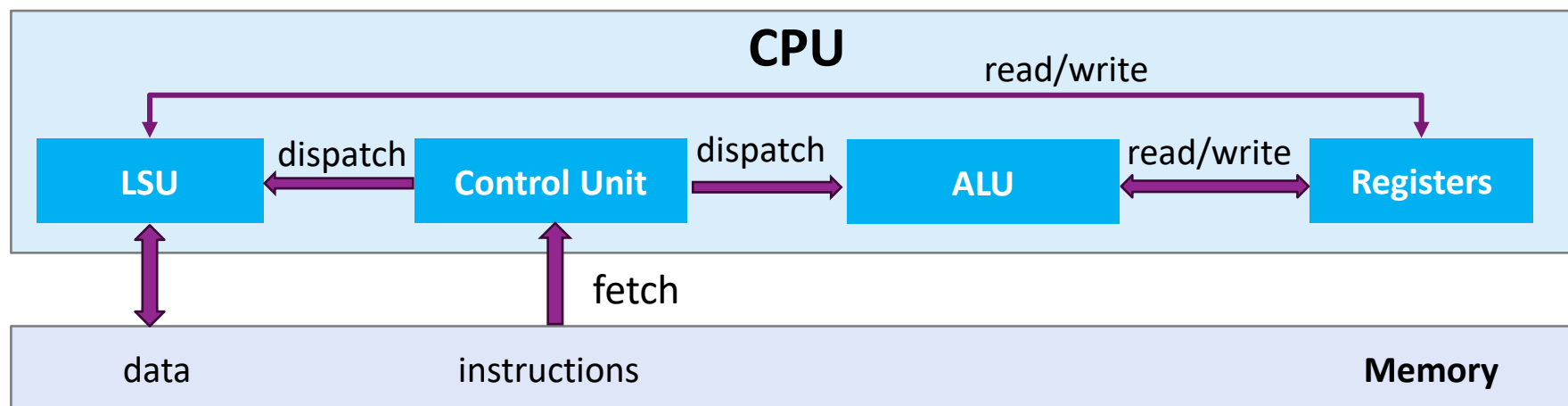
➤ x and y are two integers stored in memory.

❑ Compiling the code generates four instructions.



CPU: Central Processing Unit

- ❑ Arithmetic logic unit: Perform arithmetic/logic operations.
- ❑ Control unit: Fetch instructions from memory; decode and dispatch.
- ❑ Registers: Supply operands to the ALU and save the execution result.
- ❑ **Load-store unit: Load/store data from/to memory.**



Memory (Random-Access Memory or RAM)

- ❑ Read or write a data item based on its address in memory.
- ❑ Each data unit generally takes 1 byte or 8 bits.
- ❑ Access any data with the same latency irrespective of their locations.

	r1=load 0x18	r2=load 0x1B	r2=add r1,r2	store r2,0x20		x	y	z
--	-----------------	-----------------	-----------------	------------------	--	---	---	---

Address: 0x00 0x04 0x08 0x0B 0x10 0x14 0x18 0x1B 0x20

Memory

Memory View

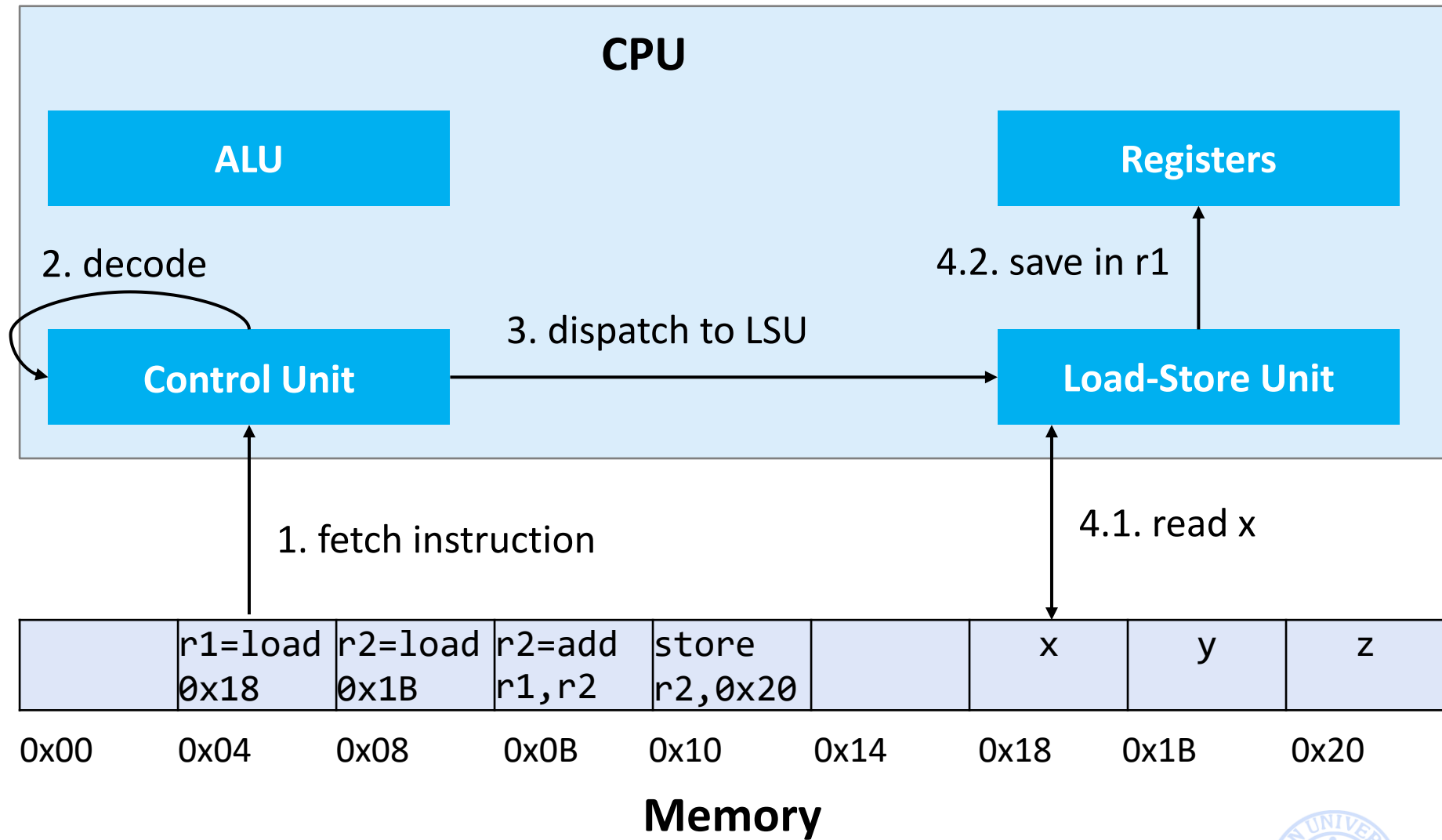
- ❑ We may assume each instruction is encoded with 4 bytes.
- ❑ The four instructions are consecutive in memory with 16 bytes.
- ❑ The program also reserves a memory slot for each variable x, y, and z.
 - x, y, and z are symbolic names.

	r1=load 0x18	r2=load 0x1B	r2=add r1,r2	store r2,0x20		x	y	z
--	-----------------	-----------------	-----------------	------------------	--	---	---	---

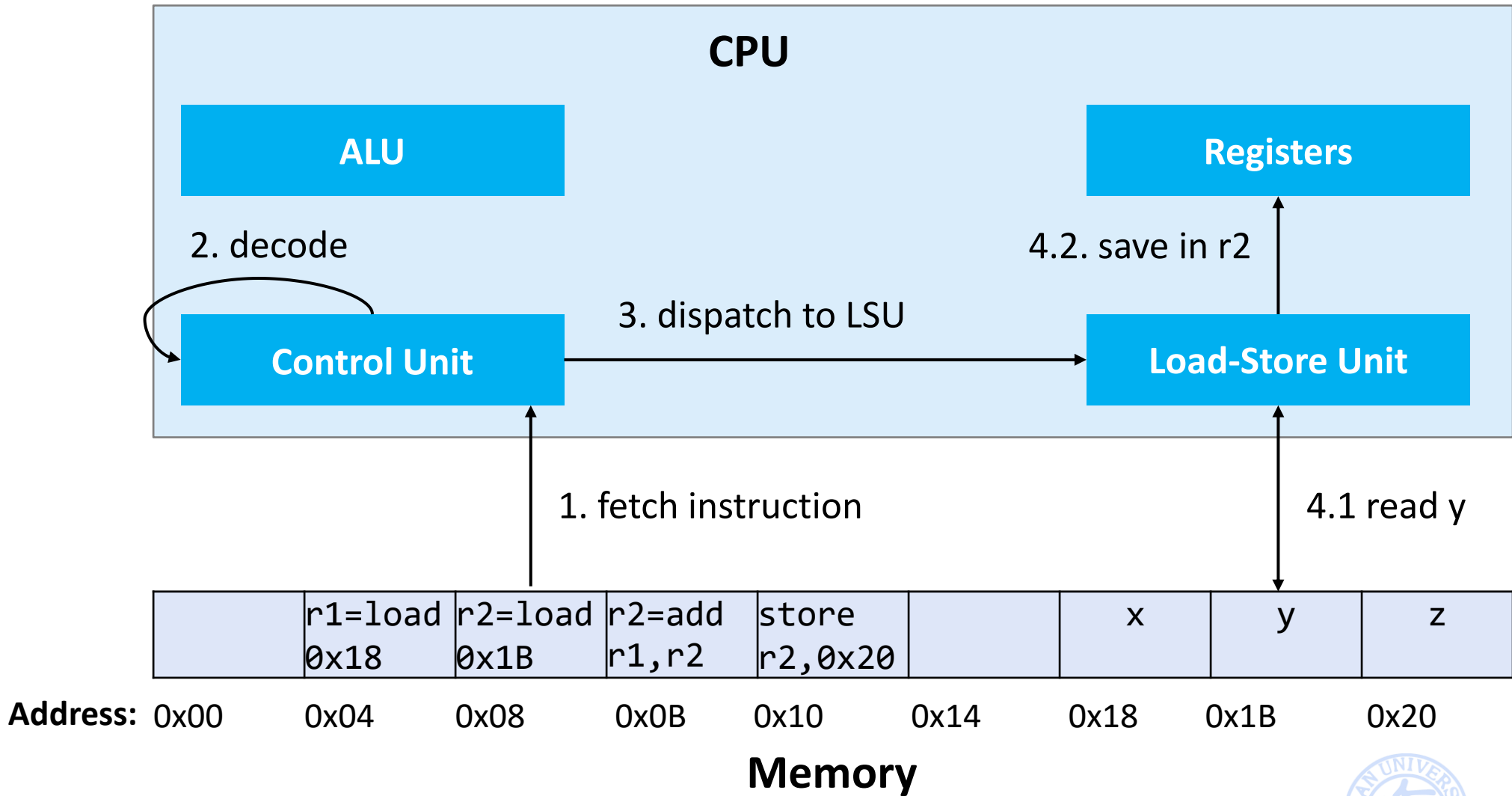
Address: 0x00 0x04 0x08 0x0B 0x10 0x14 0x18 0x1B 0x20

Memory

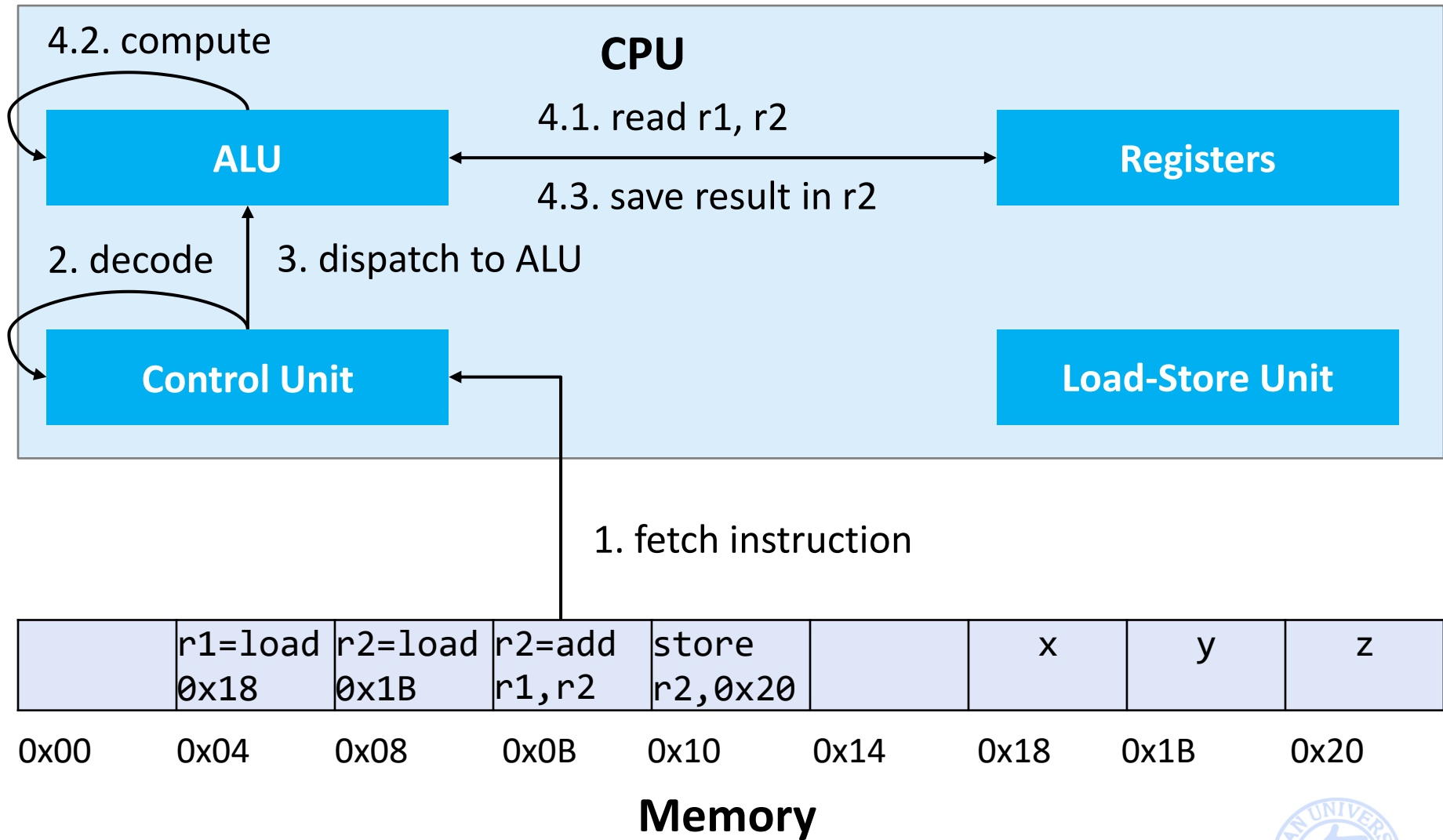
Step 1: r1 = load x



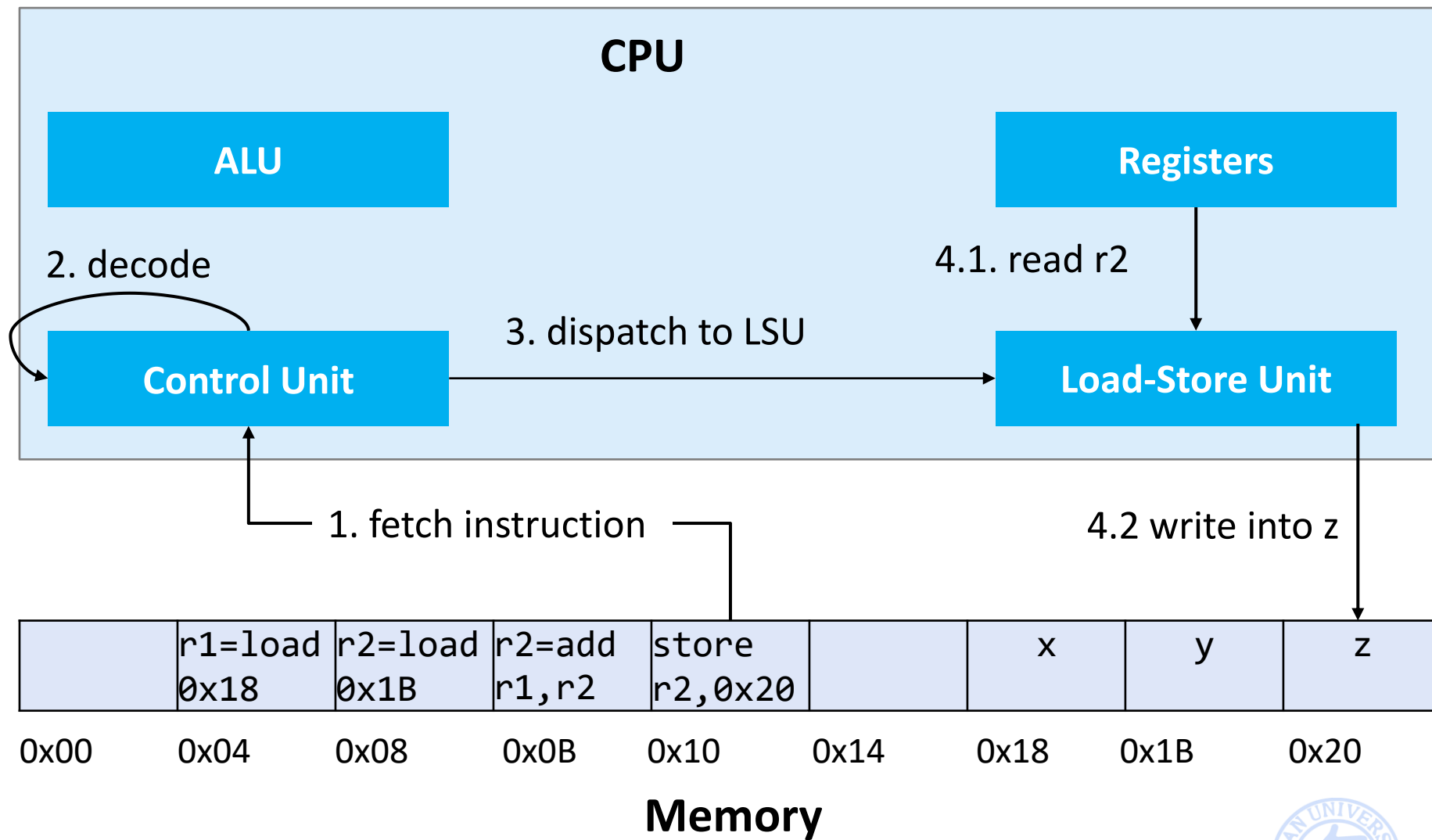
Step 2: r1 = load y



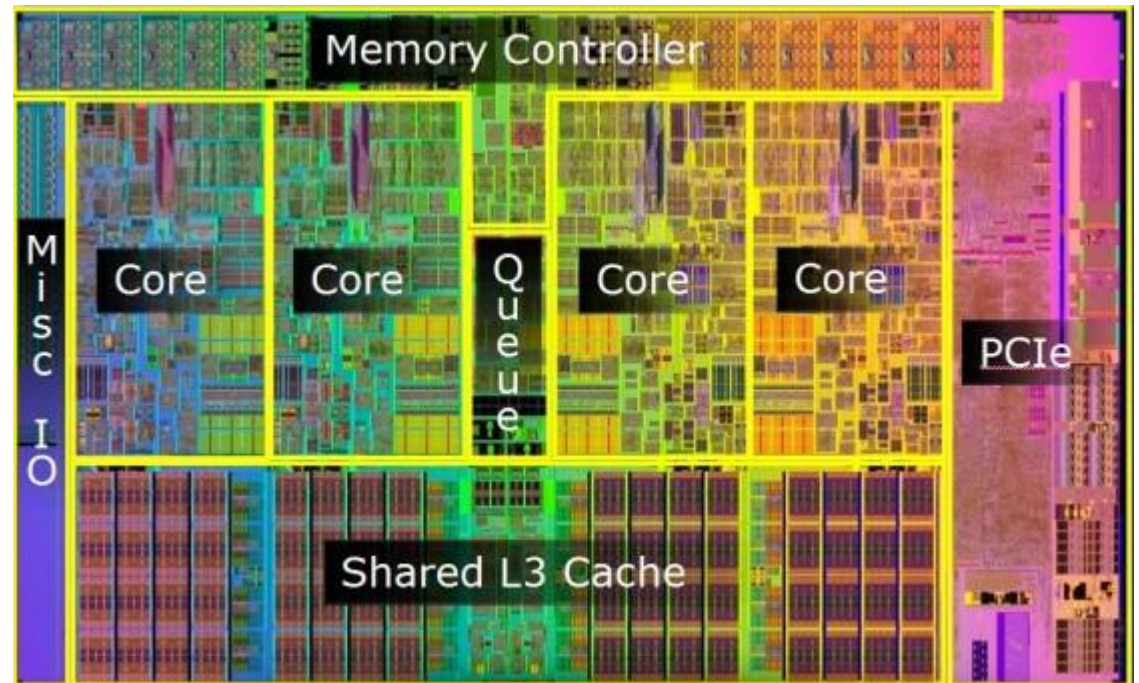
Step 3: r2=add r1, r2



Step 4: store r2, z



Intel i7 with Four Cores



Clock Speed / CPU Frequency

❑ How many clock cycles the CPU completes per second.

- Shorter clock period => Higher CPU frequency

❑ Clock ticks synchronize all operations inside the CPU

- Clock cycle is determined by the critical path of the CPU's circuits.
- Signals travel from one register → combinational logic → next register in one cycle.
- Smaller transistor size => Faster signal propagation => Shorter critical path.
- Simple instruction take 1 cycle (*e.g.*, adding two registers).
- Complex instruction may take several cycles (*e.g.*, memory access, division).

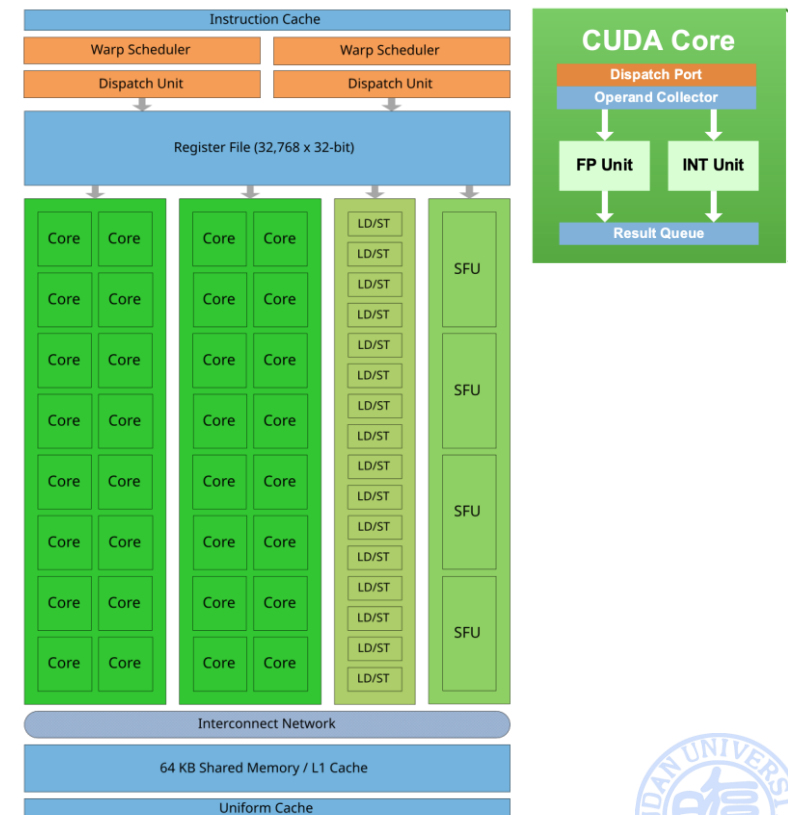
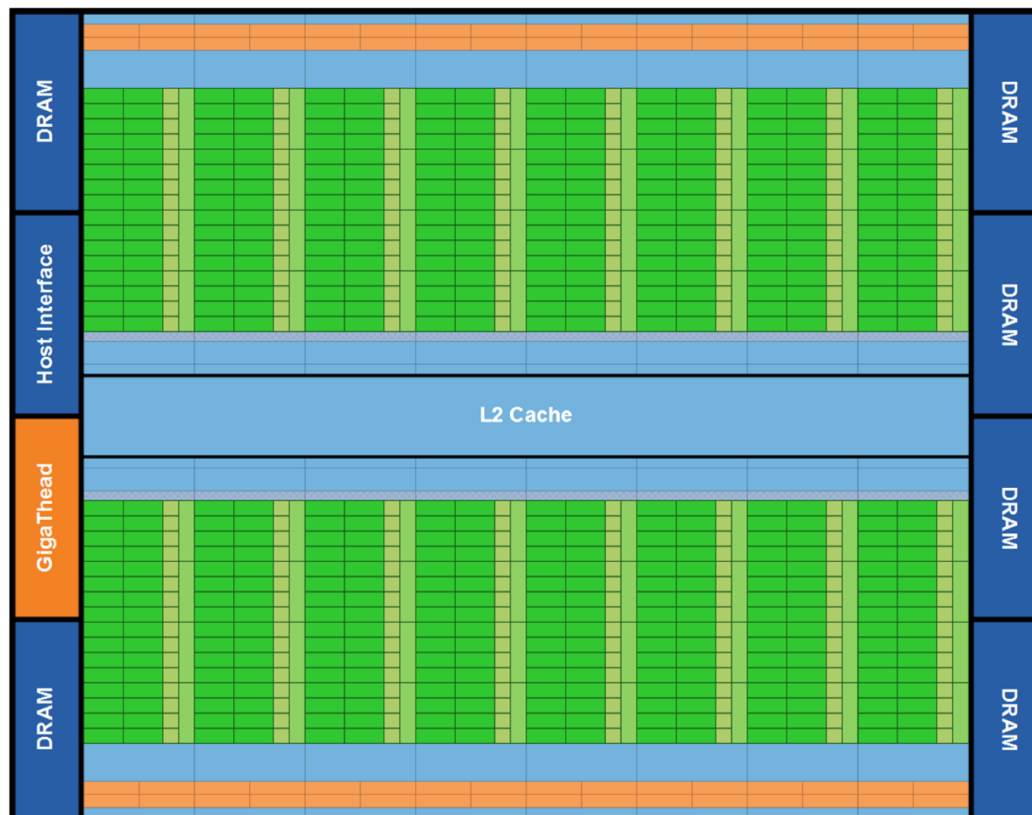
❑ Smaller transistor sizes can enable higher speed.

Question

- ❑ If a CPU employs 32 bits to represent a memory address, what is the maximum memory space the CPU can support?

General-Purpose Computing with Graphics Processing Units

- ❑ GPU has thousands of cores and a highly parallel architecture.
- ❑ It was originally designed for graphics.
- ❑ GPGPU uses GPU to perform computation typically handled by CPU.



Parameters of Different NVidia GPUs

For PC

Arch	year	Product	#. of Cores		TFLOPS (FP32)	Memory	Bandwidth GB/s	Price
			CUDA	Tensor				
Pascal	2016	GTX 1080	2560	-	8.9	8GB GDDR5X	352	\$599
Turing	2018	RTX 2080	2944	368	10.1	8GB GDDR6	448	\$699
Ampere	2020	RTX 3080	8704	328	29.8	10GB GDDR6X	912	\$699
Ada Lovelace	2022	RTX 4090	16384	512	82.6	24GB GDDR6X	1008	\$1,599
Blackwell	2025	RTX 5090	21760	680	104.8	32GB GDDR7	1792	\$1,999

For datacenter

Arch	year	Product	#. of Cores		TFLOPS (FP32)	Memory	Bandwidth GB/s	Price
			CUDA	Tensor				
Pascal	2016	P100	3584	-	9.3	16GB HBM2	732	\$5,000+
Ampere	2020	A100	6912	432	19.5	40GB HBM2e	1555	\$11,000
Hopper	2022	H100	16896	528	51.2	80GB HBM3	2039	\$30,000
Ada Lovelace	2022	L40	18176	568	90.5	48GB GDDR6	300	\$8,000
Blackwell	2024	B200	18944	592	74.45	180 HBM3E	8000	\$35,000

Exercise

❑ Check the CPU/GPU Model of Your Computer via Agent

what is the CPU model of this PC? display all the metrics

Shell Get all CPU metrics via CIM

Here are all the CPU metrics:

Property	Value
Name	Intel(R) Core(TM) i7-9700T CPU @ 2.00GHz
Manufacturer	GenuineIntel
Family	198 (Core)
Architecture	9 (x64)
Cores	8
Logical Processors	8
Thread Count	8
Current Clock Speed	1992 MHz
Max Clock Speed	1992 MHz

2. ISA and Assembly Code

ISA: Instruction Set Architecture

- ❑ **ISA is the interface that defines what a processor can do and how software communicates with the hardware.**
- ❑ **RISC: Reduced Instruction Set Computer**
 - Separate data load/store and computation into different instructions
 - *e.g.*, AArch/ARM、 RISC-V
- ❑ **CISC: Complex Instruction Set Computer**
 - An instruction can do both data load/store and computation
 - *e.g.*, X86, X86-64

Assembly Code

- ❑ Assembly code is a low-level programming language that provides a human-readable representation of machine instructions.
- ❑ Different CPU architectures have different assembly languages, such as x86-64, ARM64, and RISC-V.
- ❑ Assembly code operates directly on CPU registers and memory using instructions defined by the CPU's ISA.

ARMv8-A Instructions: Arithmetic/Logical Operations

❑ The operands of instructions can be registers or instant numbers.

❑ In the following example:

➤ x0..x5 are registers

➤ #5, #10 are instant numbers

Instruction	Semantic
<code>add x2, x0, #5</code>	$x2 = x0 + 5$
<code>sub x3, x0, x1</code>	$x3 = x0 - x1$
<code>mul x4, x0, x1</code>	$x4 = x0 * x1$
<code>sdiv x5, x0, x1</code>	$x5 = x0 / x1$
<code>and x2, x0, #5</code>	$x2 = x0 \& 5$
<code>orr x3, x0, #10</code>	$x3 = x0 \mid \mid 10$
<code>eor x4, x0, x1</code>	$x4 = x0 \wedge x1$
<code>mvn x5, x0</code>	$x5 = \sim x0$

Example:

□ How to discriminate if an integer is 2^n , where n is a positive integer?

$$x \& (x - 1) = 0$$

```
sub    x1, x0, #1  
and    x2, x0, x1  
?
```

ARMv8-A Instructions: Control Instructions

❑ Control change the normal flow of execution.

- Direct jump: The target address is specified directly in the instruction.
- Indirect jump: The target address is stored in a register.
- Conditional jump: The branch is taken only when a specified condition is satisfied.
- Function call: Branches to a function and saves the return address.

address Instructions

0x24	b	.addr1
...	...	
0x104		.addr1:

direct jump

0x24	bl	.fn1
------	-----------	------

function call

address Instructions

0x24	mov	x1, .addr1
0x28	b	x1

indirect jump

0x20	cmp	x0, x1
0x24	b.eq	addr1
...	...	
0x104		addr1:
...	...	

conditional jump

Example:

□ How to discriminate if an integer is 2^n , where n is a positive integer?

$$x \& (x - 1) = 0$$

```
sub    x1, x0, #1
and    x2, x0, x1
cmp    x2, #0
b.eq   .is_power_of_2
is_not_power_of_2:
    ...
    b followup
is_power_of_2:
    ...
followup:
    ...
```

`x0` is the variable `x`

ARMv8-A Instructions: Move

□ Move data from the source to the destination

- The source is a constant or a register.
- The destination is a register.

Instruction

```
mov x0, #5  
mov x1, x2
```

Semantic

```
x0 = 5  
x1 = x2
```

Example: Factorial

$$y = x!$$

```
    mov    x1, #1
    mov    x2, #1
loop:
    cmp    x2, x0
    b.gt   done
    mul    x1, x1, x2
    add    x2, x2, #1
    b      loop
done:
    ...
```

result = 1

i = 1

i > x?

result = result * i

i = i+1

What does the following code do?

```
    mov x1, #0
    mov x2, #2
loop:
    udiv x3, x0, x2
    mul x4, x3, x2
    cmp x4, x0
    b.eq not_p
    add x2, x2, #1
    cmp x2, x0
    b.lt loop
    b is_p
not_p:
    mov x1, #0
    b done
is_p:
    mov x1, #1
done:
    ...
```

x0 is an input variable

More ARMv8-A Instructions: Load/Store

❑ Load from/store to memory

❑ `[x]` indicates the memory address pointed by `x`

```
ldr x1, [x0]  
ldr x2, [x0, #12]  
add x3, x1, x2  
str x3, [x0, #8]
```

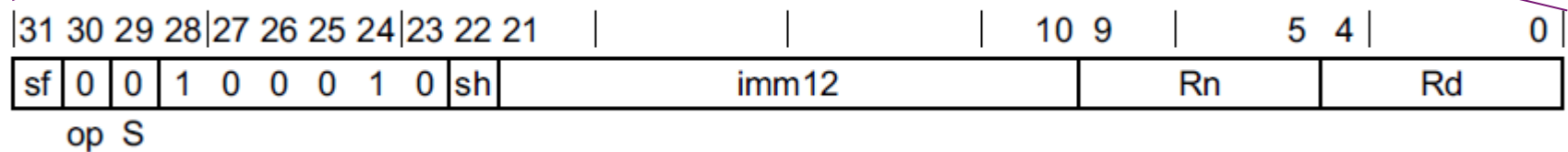
```
x1 = [x0]  
x2 = [x0 + 12]  
x3 = x1 + x2,  
[x0 + 8] = x3
```

Instructions in Memory

- Each ARMv8-A instruction has a fixed length of 32 bits.
- Limitation: Some large constants cannot be encoded directly.

ldr x0, [x2]	ldr x1, [x2, #4]	add x0, x1, #1			
-----------------	---------------------	-------------------	--	--	--

Address: 0x0..00 0x0..04 0x0..08 0x0..0B 0x0..10 0x0..14 0x0..18



add rd, rn, #4095

add rd, rn, #4097

Invalid, because 4097 needs 13 bits.



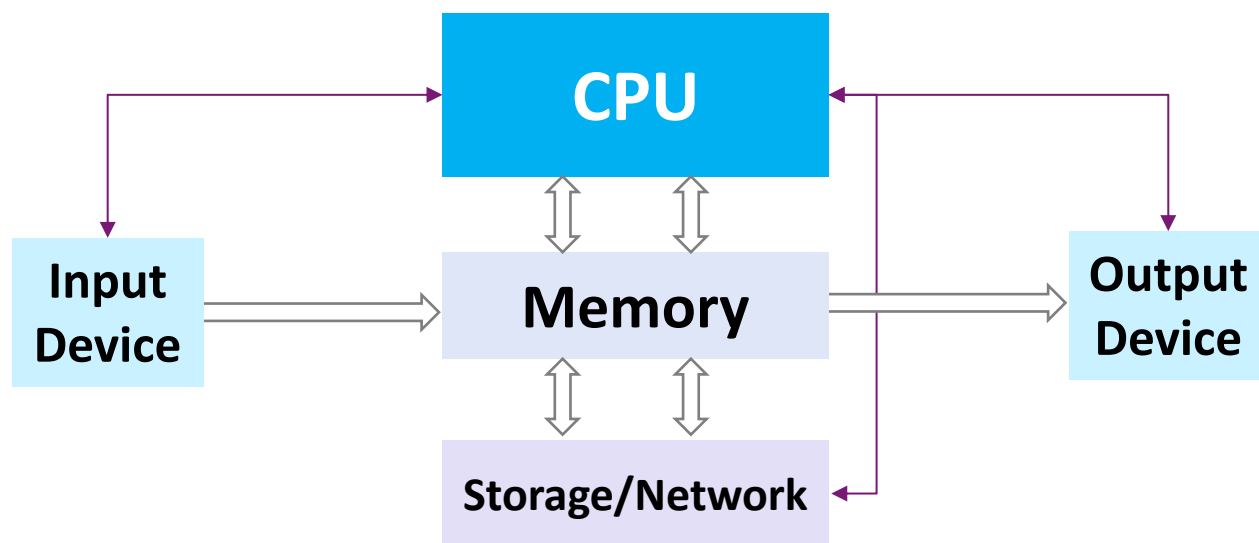
3. I/O and More

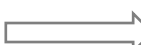
Computer Architecture

❑ Besides CPU and memory, there are peripheral devices

- Input device: keyboard, mouse, microphone, *etc.*
- Output device: display, printer, *etc.*
- Storage/network: hard disk, USB flash drive, *etc.*

❑ How does the CPU read/write peripheral devices?



control: 
data: 



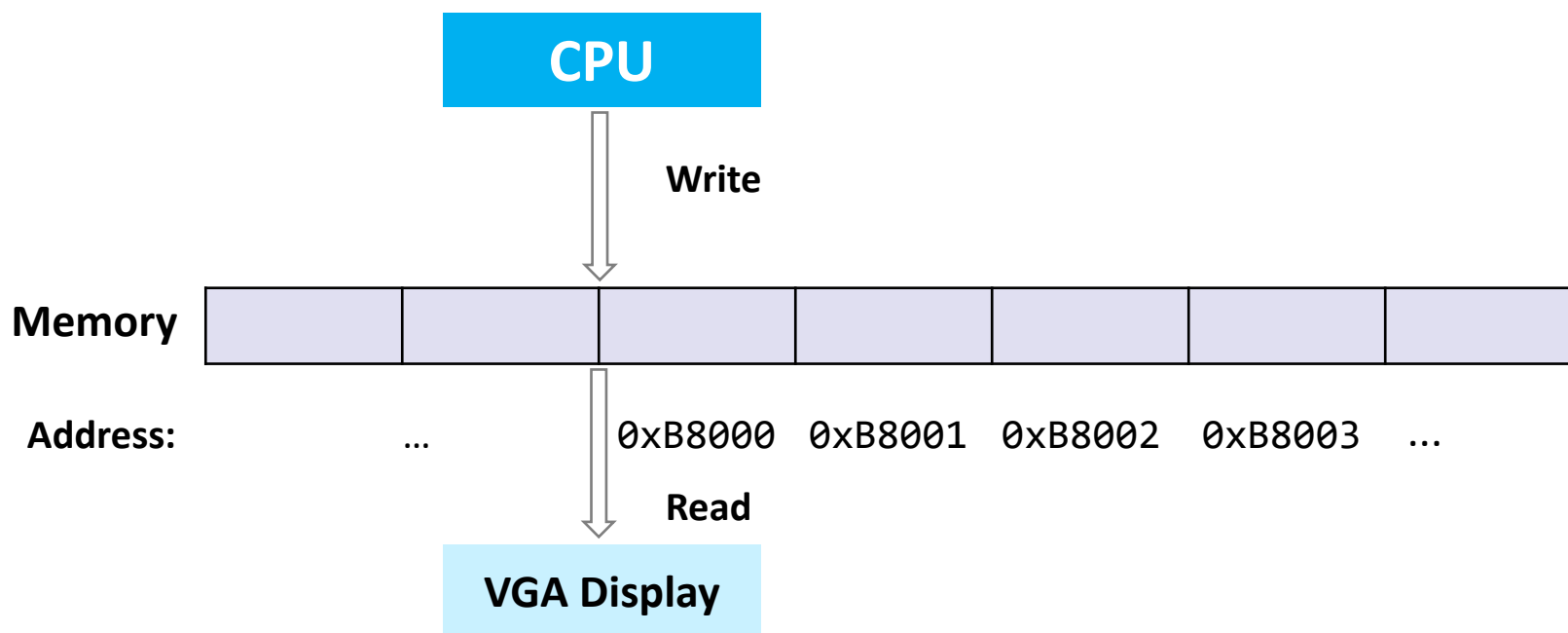
Operating System

- ❑ The operating system manages hardware resources and provides abstractions for applications.
- ❑ Applications interact with hardware through the operating system.



Memory-Mapped I/O

- ❑ Read/write data to I/O devices via memory access.
- ❑ Addressing achieved via PCI (peripheral component interconnect).



Output: Video as an Example

- ❑ **VGA (Video Graphics Array):** a standard for video display hardware.
- ❑ **The VGA memory address range on PCs is 0xA0000–0xBFFFF.**
- ❑ **0xB8000 is the starting address of color text mode (32 KB).**

```
movz    x0, #0x8000          // base address of a memory-mapped device
movk    x0, #0xB, lsl #16    // x0 = 0xB8000
mov     w1, #'A'             // character 'A'
mov     w2, #0x0F            // color
strb    w1, [x0]             // write 'A' to address 0xB8000
strb    w2, [x0, #1]         // write color to address 0xB8001
```

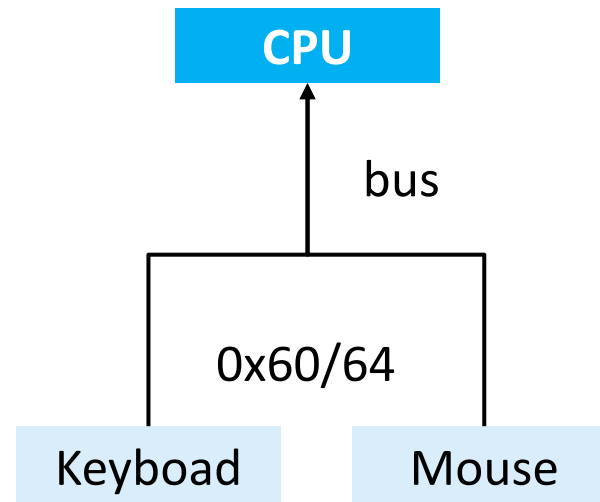
movz: Move a 16-bit immediate value into a register and clear the other bits.

movk: Replace a 16-bit portion of a register with an immediate value.

strb: Store the least significant 8 bits of a register to memory.

Alternative Approach: Port-based I/O

- ❑ Read/write data to I/O devices via port.
- ❑ For example, the PS/2 interface employs two ports:
 - 0x60 for command
 - 0x64 for data



Handling Input is More Complicated

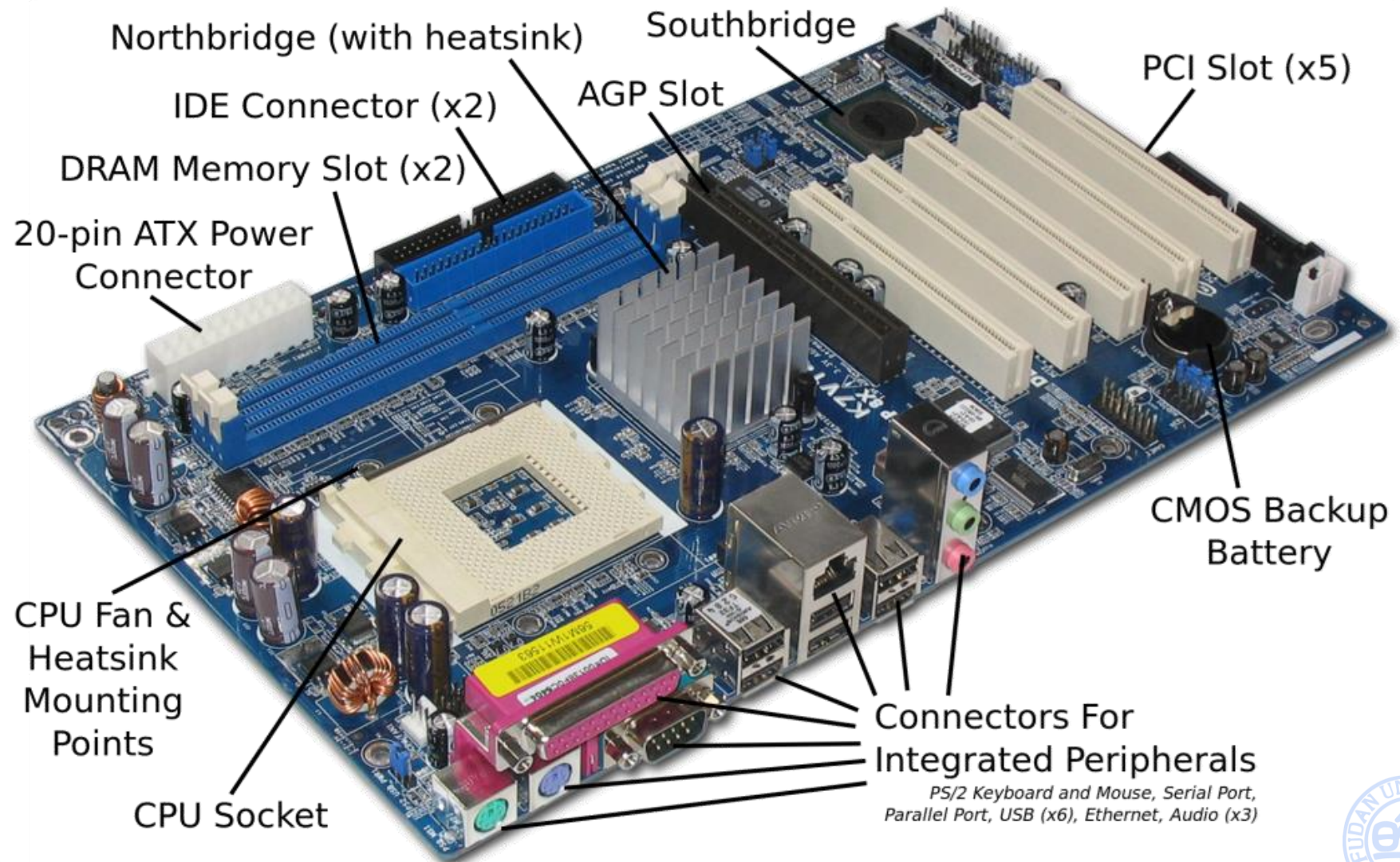
- ❑ When/how many data arrives is not controlled by the CPU.
- ❑ Polling: let the CPU check particular signals periodically.
 - How frequently?
- ❑ By interrupt (trigger-based) via particular CPU pins.



Several Concepts

- ❑ **Hardware:** Physical components of a computer system.
- ❑ **Firmware:** Specialized software embedded in hardware devices to control or manage the hardware directly.
 - *e.g.*, BIOS (basic input/output system) for interrupt handling.
- ❑ **Software:** Programs that run on the hardware.
 - *e.g.*, operating systems and applications.

Motherboard



Summary

- ❑ A CPU consists of a control unit, ALU, LSU, and registers.
- ❑ Each CPU implements an ISA and executes programs expressed as assembly instructions.
- ❑ A CPU communicates with peripheral devices via I/O ports, memory-mapped I/O, and interrupts.
- ❑ An OS manages hardware and simplifies the use of computer.

Practice with Agent

❑ Explore the hardware configuration of your computer.

➤ Try to understand the purpose and meaning of the listed parameters.

❑ What are the memory and disk capacities?

➤ Are they the same as the advertised capacities?

➤ If not, why?